

Code review and change notes

Improvement made	Why
Selected May 2026 Christchurch listings and April–June 2026 tenancy rows with overall dwelling and bedroom categories before joining.	Each area has at most one relevant tenancy row, preventing the earlier 2,911 listings from expanding to 58,455 rows.
Set the input names, output directory and period at the top of the script.	The group can see and change the assumptions without searching through the file.
Saved generated tables and charts in out/, leaving supplied inputs in data/.	Keep inputs separate from generated outputs and retain original data.
Added checks for unique listing IDs, unique tenancy area keys, join cardinality and row counts; printed unmatched listings.	A failed assumption stops the script or becomes visible rather than silently altering the analysis.
Counted distinct id values for Airbnb listings instead of distinct names.	Listings can share names; IDs identify the records.
Assigned each matched SA2 to the ward containing most of its listings before summing bonds by ward.	Avoid counting one area's bond figure more than once when listings in that area have different ward labels. This is an explicit approximation.
Labeled the ward chart as two separate counts and saved both charts before any display call.	The old script's ward plot showed the mean of area-level differences while implying ward totals; the first show() could block the second graph.

On the supplied files: 2,911 Christchurch May listings went into the left join, 2,911 rows came out, and 637 lacked a tenancy match. The matched ward chart uses 2,274 Airbnb listings across 15 wards. This validates join cardinality, not the geographic definition or suitability of the measures.

The run reported 2,911 rows before and after joining, with 637 listings that had no tenancy match. This tells us the join did not accidentally duplicate listings; it also flags the unmatched records for review.

Chart 1 — Airbnb listings and tenancy bonds by ward

“The blue bars show Airbnb listings in our May 2026 snapshot, while the orange bars show total tenancy bonds in the selected Q2 data. Central Ward has by far the most Airbnb listings and is the only ward where the blue bar is higher than the orange bar. In the other wards, the bond count is higher.”

Chart 2 — mean asking-price gap by SA2

“For each matched Airbnb listing, we subtracted one-seventh of the area’s weekly geometric mean rent from its advertised nightly price. We then averaged those gaps within each SA2. The bars are sorted from the smallest gap to the largest. Every area shown has a positive gap, although the size varies substantially.”

DATA201 Pipeline Design Principles

This document describes the pipeline implemented in Ken_D6.py for comparing Christchurch Airbnb listings with quarterly tenancy figures. The pipeline produces a checked listing-level join and two descriptive charts. Its comparisons need careful interpretation because an Airbnb listing and a tenancy bond measure different things.

1 Inputs

- `airbnb_with_area_codes.csv`: listing records including `id`, `neighbourhood_group`, `neighbourhood` (ward), `month_year`, `area_code` (SA2), and `nightly price`. The script retains Christchurch City listings for 2026-5.
- `QuarterlyTenancyCleaned.csv`: quarterly tenancy records including `timeframe`, `location_id` (SA2), `dwelling_type`, `number_of_beds`, `total_bonds`, and `geometric_mean_rent`. The script retains 2026-04-01 rows with `dwelling_type` and `number_of_beds` both equal to ALL.

Both CSV files must be in the working directory from which Ken_D6.py is run. The SA2 boundary version used for `area_code` and `location_id` still needs human confirmation.

2 Outputs

- `Ken_D6_output/merged_listings.csv`: one row per selected Airbnb listing after a left join; listings without a tenancy match retain missing tenancy values.
- `Ken_D6_output/counts_by_ward.csv`: Airbnb listing counts and tenancy bond totals for matched areas, grouped by assigned ward.
- `Ken_D6_output/difference_by_area.png`: mean listing-level nightly price gap by SA2, using Airbnb price minus geometric mean weekly rent divided by seven.
- `Ken_D6_output/bnb_vs_rental_by_ward.png`: side-by-side Airbnb listing and tenancy bond counts by assigned ward. The bars represent distinct measures, not a housing-stock ratio.

The script also prints the listing, joined-row, and unmatched counts and confirms where the files were saved.

3 Main processing steps

- Load both CSVs and create `Ken_D6_output/` for generated files.
- Filter listings to Christchurch City in May 2026 and tenancy records to the quarter beginning in April 2026 with overall dwelling and bedroom categories.
- Check that listing IDs are unique and that the selected tenancy data has one row per `location_id`. Left-join on `area_code = location_id` with many-to-one validation; check that the joined row count equals the listing count and report unmatched listings.
- For matched listings, calculate the nightly price gap and average it by SA2 for the first chart.
- Assign each matched SA2 to the ward containing the most observed listings, count distinct listing IDs, take each SA2 bond total once, then sum both counts by ward for the second chart.
- Save the joined CSV, ward summary CSV, and both PNG charts. Close the figures after saving them.

4 Coding and software strategies

- Visible assumptions: input paths, output directory, month, and quarter are named at the top of the script. A teammate can inspect or change them in one place (Week 9 Monday lecture, slides 9 and 12–13, as recorded in the group change notes).
- Assertions at risky boundaries: duplicate-key checks, merge validation, and before/after row counts stop silent many-to-many expansion. This is a concrete sanity check: 2,911 selected listings should produce 2,911 joined rows; 637 unmatched listings are reported (Monday slides 18–19 and 21).
- Meaningful identifiers and labels: distinct id values count listings; chart titles and axes name the actual measures. The ward assignment rule is stated in the code because it is an approximation (Monday slides 15–17).
- Reproducible outputs: supplied CSVs are read without modification, generated files are written to one output directory, and both plots are saved explicitly. The `main()` function keeps execution organised and prevents the pipeline from running on import (Monday slides 5–6 and 9).

Human review and AI use

The group should confirm the SA2 boundary versions, investigate unmatched listings, and decide whether the quarter and measures are suitable for its research question. The first chart compares different accommodation products; the second compares different types of counts. This document was drafted with OpenAI Codex using `Ken_D6.py`, the uploaded CSV schemas, and the group change notes. The team remains responsible for checking its accuracy before presenting it (Week 9 Tuesday lecture, slide 6, as recorded in the change notes).